Review: Modal Logic in Lean 4

Reviewer notes — April 2025

---

1. Summary

This project is a comprehensive formalization of normal modal propositional logic in Lean 4 / Mathlib, covering:

- The **syntax** of modal logic (formulas, a Hilbert-style proof system for K, derived operators).
- **Kripke semantics** (frames, models, forcing, correspondence theory, definability, undefinability).
- **Soundness and completeness** for all **16 distinct normal modal logics** obtainable from subsets of {T, B, 4, D, 5}.
- The **finite model property** (via filtration) and **decidability** of K-validity.
- A unified **modal logic cube** presentation with frame-class hierarchy theorems.

The project compiles cleanly on Lean 4.28.0 / Mathlib v4.28.0, contains **0 sorries**, uses only the standard foundational axioms (`propext`, `Classical.choice`, `Quot.sound`), and totals approximately **11,500 lines** of well-documented Lean code.

---

2. Strengths

2.1 Scope and completeness

The coverage of all 16 normal modal logics with both soundness and completeness is a notable achievement. Most formalizations in the literature cover only a handful of systems (typically K, T, S4, S5). The systematic treatment of D, 5, and their combinations — together with the collapse analysis explaining why 32 subsets reduce to exactly 16 logics — goes well beyond what is typically formalized.

2.2 Proof architecture

The project is well-structured:

- Clean separation of syntax, semantics, and metatheory.
- The canonical model construction is factored into reusable components (maximal consistent sets, Lindenbaum's lemma, truth lemma) that are instantiated per-logic.
- The Cube.lean file provides a unified birds-eye view with explicit frame-class lattice theorems.

2.3 Documentation quality

The documentation is exceptionally thorough. Nearly every file has a module docstring explaining motivation, mathematical content, and references. Individual definitions and theorems carry doc-strings. The README gives a clear overview with a table of all 16 logics.

2.4 Auxiliary results

Beyond the core soundness/completeness, the project includes:

- Filtration-based finite model property.
- Decidability of K-validity (as a `DecidablePred`).
- Local vs. global consequence distinction with concrete counterexamples.
- Undefinability of irreflexivity.
- Frame correspondence for all five axiom schemas.

2.5 Axiom audit

The `Axioms.lean` file running `#print axioms` on every major theorem is excellent practice and gives immediate confidence in the soundness of the development.

---

3. Weaknesses and Issues

3.1 Universe polymorphism

Frames are consistently used at `Frame.{0}` (universe 0). This is pragmatically fine for the current scope, but limits reusability. If the library is intended for downstream use, universe-polymorphic definitions would be more flexible.

3.2 Namespace organization

There is some namespace pollution / duplication:

- Frame class definitions live in `BasicModal` (via `Definability.lean`) but are re-exported as `abbrev` in `Modal` (via `Correspondence.lean`). This double layer adds cognitive overhead and risks divergence.
- The `Correspondence.lean` file contains two conceptual layers (syntactic definability re-exports + predicate-level correspondence) that could be more cleanly separated.

3.3 Experimental.lean

The file `Syntax/Experimental.lean` exists but its role is unclear from the project structure. If it contains work-in-progress, it should either be completed or clearly marked as non-essential and excluded from the library's public API.

3.4 Limited automation / tactics

The project does not define any custom tactics or automation for modal reasoning. All proofs are done with standard Lean/Mathlib tactics. For publishability this is fine, but for downstream usability, domain-specific tactics (e.g., a modal `decide` tactic, or automated Hilbert-system proof search) would add significant value.

3.5 Single-agent only

The project is restricted to single-agent modal logic. The README mentions a companion multi-agent project, but this project's types and definitions are not parameterized to support multi-agent generalization. If the companion project exists, cross-referencing or providing a clear upgrade path would strengthen both.

3.6 Minor style issues

- Some proof terms are somewhat long and could benefit from intermediate `have`/`let` bindings for readability.
- The `T_implies_D` proof in `Cube.lean` contains leftover comments from proof exploration (lines like "Actually, we need: ..." and "Actually, □φ ⊃ ◇φ is the D axiom schema. In T, we can derive it:"). These should be cleaned up for publication.
- A few proofs use `convert ... using 1` or `tauto` in places where more explicit reasoning would be clearer.

---

4. Publishability Assessment

4.1 Venue fit

This project is a strong candidate for publication at venues that accept formalization papers:

Venue	Fit	Notes
ITP (Interactive Theorem Proving)	★★★★★	Ideal venue. Strong formalization with novel scope.
CPP (Certified Programs and Proofs)	★★★★☆	Good fit; emphasis on certified reasoning.
Journal of Formalized Reasoning	★★★★★	Excellent fit for detailed formalization write-ups.
Journal of Automated Reasoning	★★★★☆	If the decidability/FMP results are emphasized.
Archive of Formal Proofs (Isabelle)	N/A	Wrong prover, but a Lean equivalent (e.g., Mathlib contribution) could work.

4.2 Novelty

The main novelty is **comprehensiveness**: formalizing all 16 logics in the modal cube with both soundness and completeness, plus FMP and decidability, in a single unified development. Prior formalizations of modal logic in proof assistants (Coq, Isabelle, Lean) have typically covered only a few systems. The systematic treatment of the full cube, including the D and 5 axiom interactions, appears to be new.

4.3 Readiness for publication

Nearly ready. The formalization itself is complete and solid. To prepare for submission:

1. **Write the paper.** The formalization is done; what's needed is a conference/journal paper explaining the design decisions, comparing with prior work, and highlighting the key proof techniques.
2. **Clean up minor issues** (see §3.6 above).
3. **Add a comparison table** with prior modal logic formalizations (e.g., Bentzen's Lean 4 work, From's Isabelle formalization, Doczkal & Smolka's Coq work).
4. **Consider a Mathlib contribution** for the core definitions and basic results (Formula, Frame, forcing, soundness for K).

---

5. Suggested Next Steps

Short-term (publication preparation)

1. **Clean up `T_implies_D` proof comments** and any other leftover exploration artifacts.
2. **Review `Experimental.lean`** — either complete it, remove it, or clearly document its status.
3. **Write a related-work section** comparing with existing formalizations:
   - Bentzen (Lean 4, modal logic)
   - From (Isabelle, completeness for several modal logics)
   - Doczkal & Smolka (Coq, CTL/modal logic)
   - Blanchette et al. (Isabelle, various logics)
4. **Add a `LICENSE` file** (currently only a copyright notice in the README).

Medium-term (strengthening the contribution)

5. **Prove decidability for more logics.** Currently only K-validity has a `DecidablePred` instance. Extending this to T, S4, S5 (which also have FMP) would be a natural and valuable addition. S4 and S5 have FMP via selective filtration; formalizing this would be a meaningful extension.
6. **Prove the finite model property for more logics.** The current FMP is for K only. Many of the 16 logics have FMP (in fact all of them do), and extending the filtration construction to handle frame conditions would strengthen the results significantly.
7. **Add interpolation or Beth definability.** Craig interpolation for K and several extensions is a natural next metatheoretic result.
8. **Formalize the collapse proofs syntactically.** The README explains why 32 subsets collapse to 16 logics, and some frame-theoretic collapse results are formalized, but formalizing the *syntactic* collapses (e.g., showing that T proves D, that T+5 proves B and 4) would complete the picture.

Long-term (library development)

9. **Contribute core definitions to Mathlib.** The `Form`, `Frame`, `forces`, and basic soundness/completeness results for K would be a valuable Mathlib contribution, establishing a standard modal logic API.
10. **Integrate with the multi-agent companion project.** Parameterize the formula type by agent and generalize the metatheory.
11. **Add temporal logic.** The framework naturally extends to basic temporal logics (LTL, CTL) by reinterpreting the accessibility relation.
12. **Custom tactics.** A `modal_decide` tactic that combines the decidability result with proof-by-reflection could make the library much more usable for downstream applications.

---

6. Overall Assessment

This is a **high-quality, publication-ready formalization** that makes a genuine contribution to the formal verification of modal logic metatheory. The scope (16 logics, soundness + completeness + FMP + decidability), the clean architecture, and the thorough documentation all meet the standards of top formalization venues. The main work remaining is writing the accompanying paper and performing minor cleanup.

Recommendation: Publish. Target ITP or the Journal of Formalized Reasoning.